

EXP.1. Write program demonstrates how to use regular expressions in Python to match and search for patterns in text.

CODE:

```
import re

text = input("Enter a sentence: ")

phone = re.findall(r"\d{10}", text)

email = re.findall(r"\S+@\S+", text)

print("Phone Number:", phone)

print("Email:", email)

word = input("Enter the word to search: ")

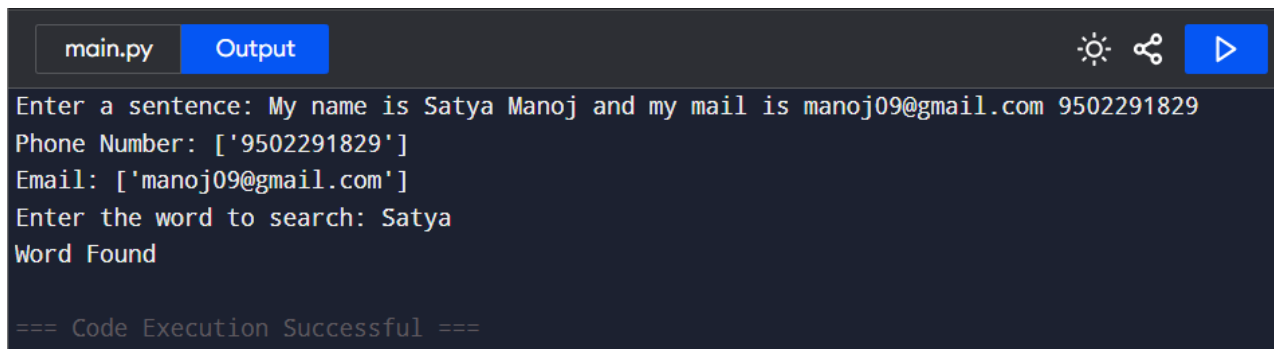
if re.search(word, text):

    print("Word Found")

else:

    print("Word Not Found")
```

Output:

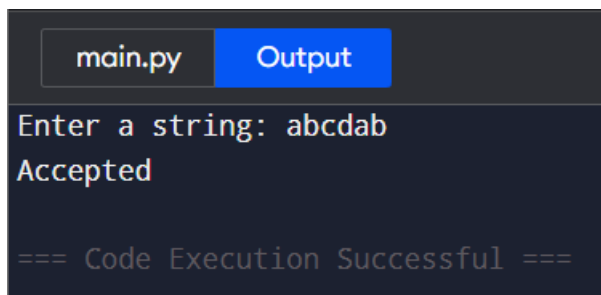
A screenshot of a Python code execution environment. At the top, there are two tabs: 'main.py' and 'Output'. The 'Output' tab is active, showing the results of the program's execution. The text in the output window is: 'Enter a sentence: My name is Satya Manoj and my mail is manoj09@gmail.com 9502291829', 'Phone Number: ['9502291829']', 'Email: ['manoj09@gmail.com']', 'Enter the word to search: Satya', 'Word Found', and '=== Code Execution Successful ==='. The interface includes standard icons for settings, sharing, and running code.

EXP.2. Implement a basic finite state automaton that recognizes a specific language or pattern. In this example, we'll create a simple automaton to match strings ending with 'ab' using python.

CODE:

```
def finite_state_automaton(string):  
    return string.endswith("ab")  
  
s = input("Enter a string: ")  
  
if finite_state_automaton(s):  
    print("Accepted")  
else:  
    print("Rejected")
```

Output:



```
main.py Output  
Enter a string: abcdab  
Accepted  
  
=== Code Execution Successful ===
```

EXP.3. Write program demonstrates how to perform morphological analysis using the NLTK library in Python.

CODE:

```
import nltk

from nltk.tokenize import word_tokenize

from nltk.stem import WordNetLemmatizer

nltk.download('punkt')

nltk.download('wordnet')

nltk.download('omw-1.4')

nltk.download('punkt_tab') # Added to resolve LookupError

text = input("Enter a sentence: ")

words = word_tokenize(text)

lemmatizer = WordNetLemmatizer()

print("Word\tLemma")

for word in words:

    print(word, "\t", lemmatizer.lemmatize(word))
```

Output:

```
... [nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
[nltk_data] Package omw-1.4 is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
Enter a sentence: The cats are playing with the balls
Word      Lemma
The       The
cats      cat
are       are
playing   playing
with      with
the       the
balls     ball
```

EXP.4 Implement a finite-state machine for morphological parsing. In this example, we'll create a simple machine to generate plural forms of English nouns using python.

CODE:

```
def generate_plural(noun):  
    if noun.endswith(("s", "x", "z", "ch", "sh")):  
        plural = noun + "es"  
    elif noun.endswith("y") and len(noun) > 1 and noun[-2].lower() not in "aeiou":  
        plural = noun[:-1] + "ies"  
    else:  
        plural = noun + "s"  
    return plural  
  
words = ["Lion", "dog", "bus", "bat", "church",  
        "dish", "children", "city", "toy", "book"]  
  
print("Singular\tPlural")  
print("-----")  
for word in words:  
    print(f'{word:10}\t{generate_plural(word)}')
```

Output:

...	Singular	Plural

	Lion	Lions
	dog	dogs
	bus	buses
	bat	bats
	church	churches
	dish	dishes
	children	childrens
	city	cities
	toy	toys
	book	books

EXP.5. Use the Porter Stemmer algorithm to perform word stemming on a list of words using python libraries.

CODE:

```
import nltk

from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

word = input("Enter a word: ")

stem = stemmer.stem(word)

# Display result

print("\nOriginal Word :", word)

print("Stemmed Word :", stem)
```

OUTPUT:

```
... Enter a word: Playing

Original Word : Playing
Stemmed Word  : playing
```

EXP.6. Implement a basic N-gram model for text generation. For example, generate text using a bigram model using python.

CODE:

```
import nltk

from nltk import bigrams

from collections import defaultdict

import random

nltk.download('punkt')

nltk.download('punkt_tab')

text = input("Enter a sentence: ")

words = nltk.word_tokenize(text.lower())

model = defaultdict(list)

for w1, w2 in bigrams(words):

    model[w1].append(w2)

start = input("Enter the starting word: ").lower()

if start not in model:

    print("Starting word not found in the sentence.")

else:

    generated = [start]

    current = start

    for i in range(10):

        if current not in model:

            break

        current = random.choice(model[current])

        generated.append(current)

    print("\nGenerated Text:")

    print(" ".join(generated))
```

Output:

```
... [nltk_data] Downloading package punkt to /root/nltk_data...  
[nltk_data]   Package punkt is already up-to-date!  
[nltk_data] Downloading package punkt_tab to /root/nltk_data...  
[nltk_data]   Package punkt_tab is already up-to-date!  
Enter a sentence: My name is Satya Manoj  
Enter the starting word: My  
  
Generated Text:  
my name is satya manoj
```